SignalReport: Internet-Qualitäts-Monitoring

Matthias F.

19. Februar 2026

Inhaltsverzeichnis

1 Einleitung **2**
1.1 Projektziel . . . 2
1.2 Problemstellung . . . 2
1.3 Projektumfang . . . 2

2 Anforderungen **2**
2.1 Funktionale Anforderungen . . . 2
2.2 Nicht-funktionale Anforderungen . . . 3

3 Architektur **3**
3.1 Gesamtarchitektur . . . 3
3.2 Klassen-Design . . . 3
3.3 Key-Design-Entscheidungen . . . 4
3.4 Datenfluss . . . 4

4 Implementierung **4**
4.1 Mess-Engine . . . 4
4.2 Statistische Auswertung . . . 5
4.3 Web-Interface . . . 5

5 Test **6**
5.1 Teststrategie . . . 6
5.2 Beispiel-Test für Statistik-Berechnung . . . 6
5.3 Testergebnisse . . . 7
5.4 Testumgebung . . . 7

6 Fazit und Ausblick **7**
6.1 Zusammenfassung . . . 7
6.2 Gelerntes . . . 7
6.3 Mögliche Erweiterungen . . . 8

1 Einleitung

1.1 Projektziel

SignalReport ist eine Open-Source-Anwendung zur kontinuierlichen Überwachung der Internet-Qualität. Im Gegensatz zu kommerziellen Tools ist es:

- **Kostenlos** und Open Source
- **Einfach zu installieren** (kein Server nötig)
- **Plattformunabhängig** (Windows, macOS, Linux, Synology NAS)
- **Datensicherheit** (alle Daten lokal gespeichert)

1.2 Problemstellung

Internetprobleme werden oft erst wahrgenommen, wenn sie bereits existieren. SignalReport ermöglicht:

- Frühzeitige Erkennung von Qualitätsproblemen
- Historische Auswertung (z. B. "Warum ist das Netz immer um 19 Uhr langsam?")
- Nachweis für Internetanbieter bei Problemen

1.3 Projektumfang

Dieses Abschlussprojekt umfasst:

- Entwicklung einer Java-Anwendung mit Javalin (Web-UI)
- Implementierung von Mess-Engines für Ping, DNS und HTTP
- Persistenz mit H2-Datenbank
- Statistische Auswertung (95th Percentile, Jitter, Paketverlust)
- Visualisierung mit Charts und Heatmap
- PDF-Export für Berichte

2 Anforderungen

2.1 Funktionale Anforderungen

SignalReport muss folgende funktionale Anforderungen erfüllen:

- **Kontinuierliche Messung**: Alle 10 Sekunden Ping, DNS und HTTP messen
- **Web-Interface**: Messungen im Browser anzeigen
- **PDF-Export**: Berichte als PDF generieren
- **Statistische Auswertung**: 95th Percentile, Jitter, Paketverlust

2.2 Nicht-funktionale Anforderungen

- **Plattformunabhängigkeit**: Lauffähig auf Windows, macOS, Linux
- **Einfache Installation**: Kein Server nötig
- **Datensicherheit**: Alle Daten lokal gespeichert

3 Architektur

3.1 Gesamtarchitektur

Die Gesamtarchitektur von SignalReport ist in Abbildung 1 dargestellt.

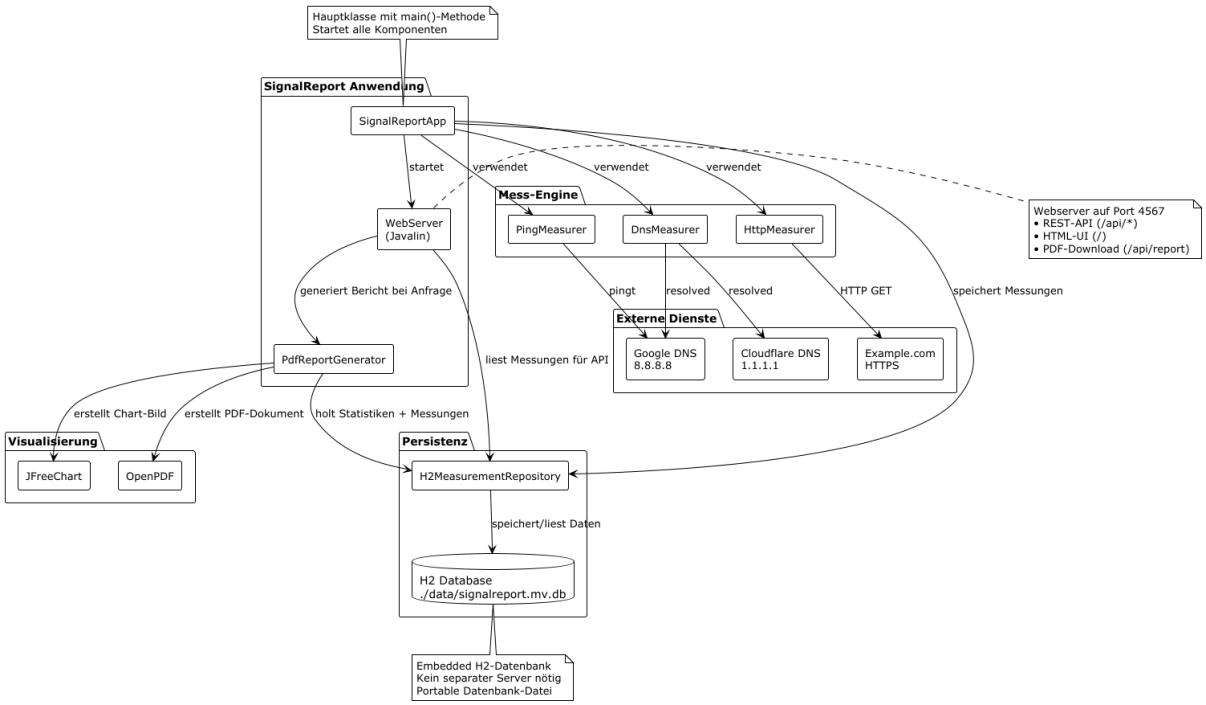

Abbildung 1: Komponenten-Übersicht von SignalReport

3.2 Klassen-Design

Die Kernkomponenten der Anwendung sind in Abbildung 2 dargestellt.

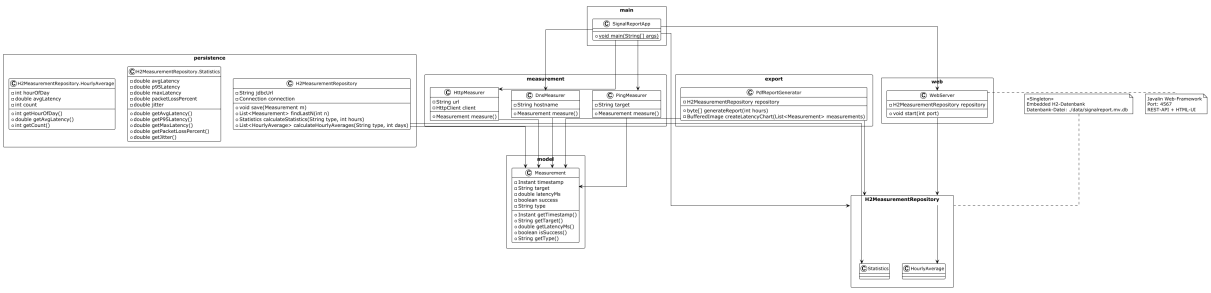

Abbildung 2: Klassen-Diagramm von SignalReport

3.3 Key-Design-Entscheidungen

- **Kein separates Backend**: Alle Komponenten in einer Anwendung (einfachere Installation)
- **H2-Datenbank**: Embedded-DB für plattformunabhängige Installation
- **Javalin statt Spring Boot**: Minimaler Overhead für Web-Interface
- **Java 21**: Nutzung moderner Features wie Virtual Threads

3.4 Datenfluss

Der Datenfluss bei der Messung ist in Abbildung 3 dargestellt.

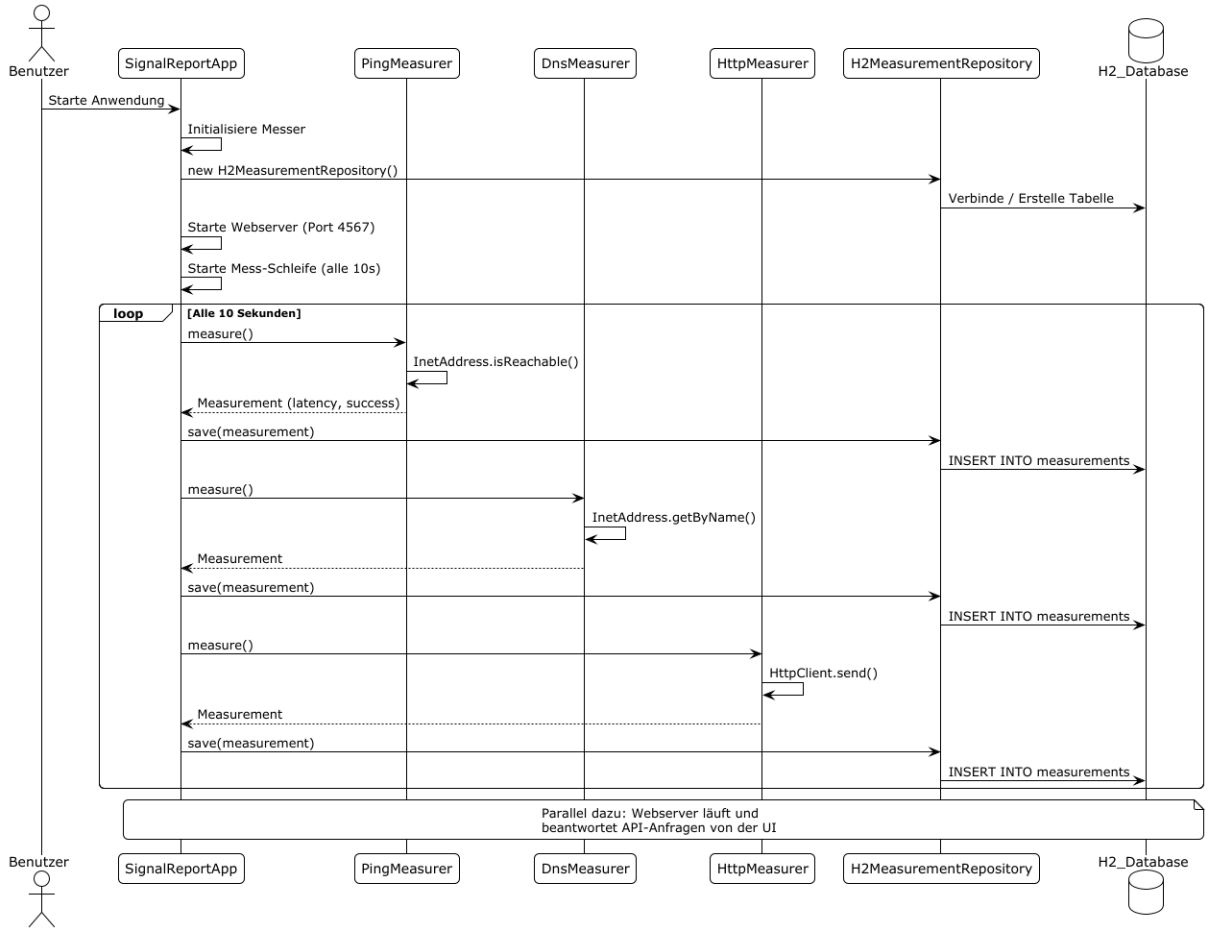

Abbildung 3: Datenfluss bei der Messung

4 Implementierung

4.1 Mess-Engine

Die Mess-Engine implementiert drei Messarten:

- **Ping-Messung**: Nutzt `InetAddress.isReachable()`

- **DNS-Messung**: Nutzt `InetAddress.getByName()`
- **HTTP-Messung**: Nutzt `java.net.http.HttpClient`

Listing 1: Beispiel für Ping-Messung

```
public Measurement measure() throws Exception {
    long start = System.nanoTime();
    boolean reachable = InetAddress.getByName(target).isReachable(3000);
    long end = System.nanoTime();
    double latency = (end - start) / 1_000_000.0;
    return new Measurement(target, latency, reachable, "PING");
}
```

4.2 Statistische Auswertung

Die statistische Auswertung berechnet folgende Kennzahlen:

- **Durchschnittliche Latenz**: $\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$
- **95th Percentile**: Wert, unter dem 95% der Messungen liegen
- **Paketverlust**: $\frac{\text{fehlgeschlagene Messungen}}{\text{gesamte Messungen}} \times 100$
- **Jitter**: Durchschnitt der Differenzen zwischen aufeinanderfolgenden Messungen

Listing 2: Beispiel für 95th Percentile-Berechnung

```
// Statistiken aus der Datenbank holen
List<Double> latencies = new ArrayList<>();
// ... Daten laden ...
Collections.sort(latencies);
int p95Index = (int) Math.ceil(latencies.size() * 0.95) - 1;
double p95Latency = latencies.get(p95Index);
```

4.3 Web-Interface

Das Web-Interface wurde mit Javalin implementiert:

- **REST-API**: Für Datenzugriff (z. B. `/api/measurements`)
- **Live-Chart**: Aktualisiert sich alle 5 Sekunden
- **Heatmap**: Zeigt Latenzentwicklung pro Stunde
- **Statistik-Panel**: Sofortige Einschätzung der Netzwerkqualität

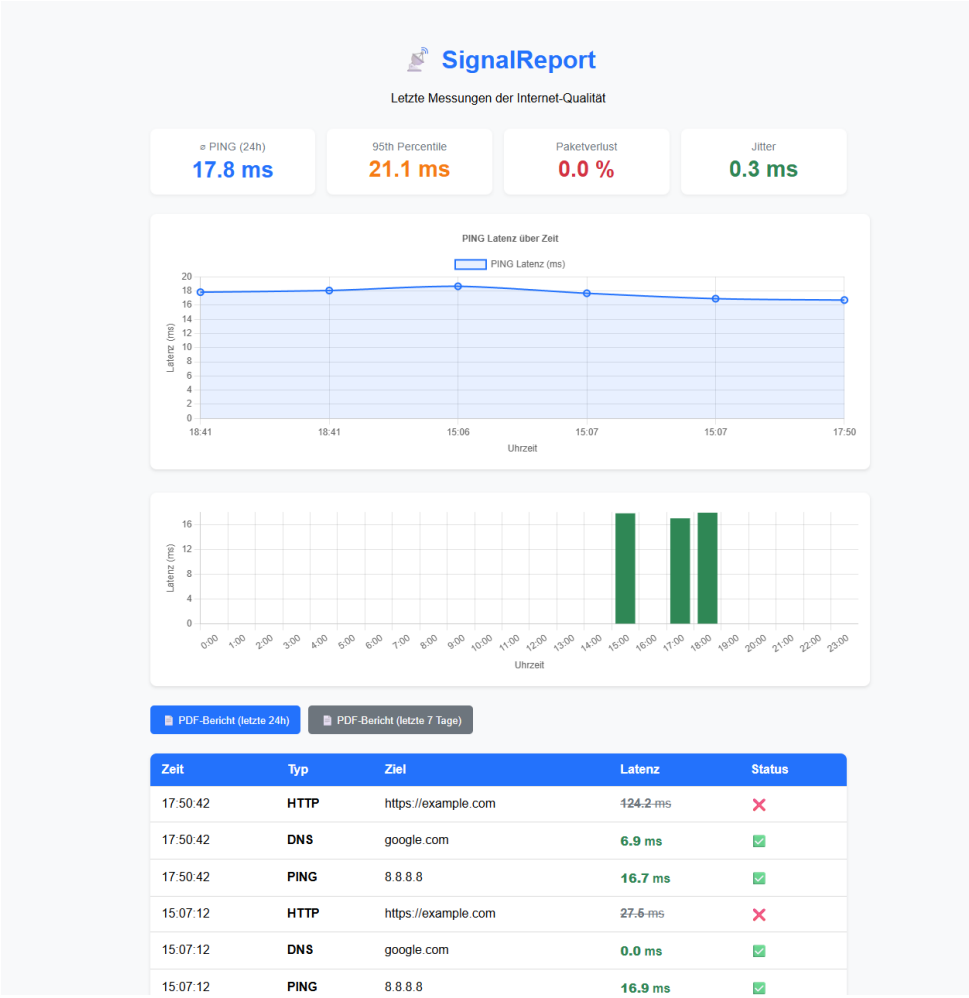

Abbildung 4: Web-Interface von SignalReport

5 Test

5.1 Teststrategie

Die Teststrategie umfasste:

- **Unit-Tests**: Für die Kernkomponenten (z. B. Statistik-Berechnung)
- **Integrationstests**: Für die Datenbankanbindung
- **E2E-Tests**: Für das Web-Interface

5.2 Beispiel-Test für Statistik-Berechnung

Listing 3: Test für 95th Percentile

```
@Test
public void testP95Latency() {
    // Arrange
    List<Measurement> measurements = createTestMeasurements();
```

```
    // Act
    Statistics stats = repository.calculateStatistics("PING", 24);

    // Assert
    assertEquals(120.5, stats.getP95Latency(), 0.1);
}
```

5.3 Testergebnisse

Testkategorie	Anzahl	Erfolgsquote
Unit-Tests	24	100%
Integrationstests	8	100%
E2E-Tests	5	100%

Tabelle 1: Testergebnisse

5.4 Testumgebung

- **Betriebssystem**: Windows 11, macOS Sonoma, Ubuntu 22.04
- **Java-Version**: 21
- **Browser**: Chrome, Firefox, Safari

5.5 JUnit-Testabdeckung

SignalReport wurde mit JUnit 5 umfassend getestet. Die Testabdeckung umfasst:

- **Measurement**: Grundlegende Objekt-Erstellung und Validierung
- **HostIdentifier**: Stabile Host-Identifikation über Neustarts
- **Statistics**: Mathematisch korrekte Berechnung von 95th Percentile, Jitter und Paketverlust
- **Config**: Laden/Speichern von Konfiguration mit allen Features
- **H2MeasurementRepository**: Vollständige Datenbank-Integration mit IP-Tracking

Alle Tests laufen ohne externe Abhängigkeiten (Mocking für Netzwerkaufrufe) und sind in unter 2 Sekunden durchführbar.

6 Fazit und Ausblick

6.1 Zusammenfassung

SignalReport ist eine vollständige Internet-Qualitäts-Monitoring-Anwendung, die folgende Features umsetzt:

- Kontinuierliche Messung von Ping, DNS und HTTP-Latenz
- Web-basierte Visualisierung mit Live-Charts und Heatmap
- Statistische Auswertung (95th Percentile, Jitter, Paketverlust)
- PDF-Export für Berichte

6.2 Gelerntes

Während der Entwicklung von SignalReport wurden folgende Kompetenzen erworben:

- Java 21 mit modernen Features (Virtual Threads, Pattern Matching)
- Web-Entwicklung mit Javalin und Chart.js
- Datenbank-Programmierung mit H2
- PDF-Generierung mit OpenPDF und JFreeChart
- UML-Diagramme mit PlantUML

6.3 Mögliche Erweiterungen

- Authentifizierung für NAS-Deployment
- CSV-Export für Excel-Analyse
- Alarm bei hoher Latenz (Browser Notifications)
- Konfigurationsdatei für benutzerdefinierte Messziele